UNITÀ 2 · WEB SERVICE · LEZIONE 10

Usare REST da Java e JSP

Il server c'è. Ora dall'altra parte del filo: un programma Java che chiama BiblioAPI e una pagina JSP che trasforma il suo JSON in una pagina web.

5G TPSI HttpClient, POST, errori di rete, chiamate asincrone, JSP su Tomcat

PER COMINCIARE

Una pagina deve mostrare i libri di BiblioAPI. Chi dovrebbe chiamare l'API?

SONDAGGIO

- **A** Il browser, con JavaScript
- **B** Il server web, che poi manda l'HTML pronto
- **C** Dipende da chi deve vedere i dati
- **D** Non importa, è uguale

Una GET in tre oggetti

```
import java.net.URI;
import java.net.http.*;
import java.time.Duration;
import java.util.regex.*;

public class ClientBiblio {
  public static void main(String[] args) throws Exception {
    HttpClient client = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(3)).build();
    HttpRequest get = HttpRequest.newBuilder(URI.create("http://localhost:8000/api/libri"))
        .header("Accept", "application/json").GET().build();
    HttpResponse<String> r = client.send(get, HttpResponse.BodyHandlers.ofString());
    System.out.println("Stato: " + r.statusCode());
    Matcher m = Pattern.compile("\"titolo\":\"([^\"]*)\"").matcher(r.body());
    while (m.find()) System.out.println("- " + m.group(1));
  }
}
```

AL COMPUTER · 15 minuti

Il primo client di BiblioAPI

1. Avvia BiblioServer della lezione 9 in un terminale.
2. In un'altra cartella, **client**, crea **ClientBiblio.java** con il codice della slide.
3. In un secondo terminale: `java ClientBiblio.java`.
4. Stampa anche l'header `Content-Type`: `r.headers().firstValue("Content-Type")`.
5. Cambia l'URI in `/api/libri/9`: che codice ottieni? E che eccezione?

Controllo: `Stato: 200` e i titoli dei tre libri, uno per riga. L'header vale `Optional[application/json; charset=utf-8]`. Con `/9`: `Stato: 404` e **nessuna eccezione**.

PUZZLE

Abbina ogni pezzo del client al suo compito

HttpClient	---	Dice come leggere il corpo della risposta
HttpRequest	---	Trasforma una stringa nel corpo della richiesta
BodyPublishers.ofString	---	Si crea una volta e si riusa: gestisce le connessioni
BodyHandlers.ofString	---	Immutabile: URI, metodo, header, corpo
sendAsync	---	Restituisce subito un CompletableFuture

INVIARE DATI

POST con JSON, e un limite di tempo

```
String json = """
    {"titolo":"Lessico famigliare","autore":"Natalia Ginzburg","anno":1963}""";
HttpRequest post = HttpRequest.newBuilder(URI.create(BASE))
    .header("Content-Type", "application/json")
    .timeout(Duration.ofSeconds(5))
    .POST(HttpRequest.BodyPublishers.ofString(json))
    .build();
try {
  HttpResponse<String> c = client.send(post, HttpResponse.BodyHandlers.ofString());
  System.out.println(c.statusCode() + " " + c.headers().firstValue("Location").orElse("-"));
} catch (java.net.ConnectException e) {
  System.out.println("BiblioAPI non raggiungibile: il server è acceso?");
} catch (java.net.http.HttpTimeoutException e) {
  System.out.println("Nessuna risposta entro 5 secondi");
}
```

QUIZ A TRABOCCHETTO

Il server è spento. Che cosa succede a `client.send(get, …)`?

QUIZ

A Restituisce una risposta con codice 404

B Lancia una ConnectException: non c'è nessun codice di stato

C Restituisce una risposta con codice 503

D Restituisce null

AL COMPUTER · 25 minuti

Un client che non crolla

1. Aggiungi a ClientBiblio il POST della slide, con i due `catch`.
2. Dopo il POST, fai una GET sull'URI ricevuto in `Location` e stampa il JSON.
3. Se il codice è 4xx o 5xx, stampa `Errore` seguito dal corpo della risposta.
4. Spegni il server (Ctrl+C) e riesegui il client.
5. Salva il file finale: lo consegnerai con il compito.

Controllo: a server acceso: `201 /api/libri/4` e poi il JSON di Lessico famigliare. Riesegui: `/api/libri/5`. A server spento: `BiblioAPI non raggiungibile: il server è acceso?` e nessuno stack trace.

SENZA ASPETTARE

Più richieste insieme con sendAsync

```
var f1 = client.sendAsync(richiesta("/1"),
                  BodyHandlers.ofString());
var f2 = client.sendAsync(richiesta("/2"),
                  BodyHandlers.ofString());
// qui il programma può fare altro
String t1 = f1.thenApply(HttpResponse::body).join();
String t2 = f2.thenApply(HttpResponse::body).join();
System.out.println(t1 + "\n" + t2);
```

- `send` blocca il thread finché la risposta non arriva.
- `sendAsync` torna subito con un **CompletableFuture**: le richieste viaggiano in parallelo.
- `thenApply` dice che cosa fare con la risposta; `join` aspetta il risultato.
- Tempo totale ≈ quello della risposta più lenta, non la somma.

JAVASERVER PAGES

HTML con dentro un po' di Java, eseguito sul server

Direttiva

`<%@ page import="…" %>`

Impostazioni della pagina: import, codifica.

Scriptlet

`<% codice Java %>`

Istruzioni eseguite a ogni richiesta.

Espressione

`<%= valore %>`

Il valore finisce nell'HTML.

> Alla prima richiesta Tomcat **traduce la JSP in una servlet** Java e la compila. Il browser non vede mai il Java: riceve solo l'HTML prodotto.

Nei progetti veri il Java sta in una servlet e la JSP mostra soltanto i dati (modello MVC). Per il laboratorio basta una pagina.

LA PAGINA DELLA BIBLIOTECA

libri.jsp: l'API dentro una tabella

```
<%@ page contentType="text/html; charset=UTF-8" import="java.net.URI, java.net.http.*, java.util.regex.*" %>
<%! static final HttpClient CLIENT = HttpClient.newHttpClient();
    static String html(String s) {
      return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;").replace("\"", "&quot;");
    } %>
<% HttpRequest req = HttpRequest.newBuilder(URI.create("http://localhost:8000/api/libri")).build();
   HttpResponse<String> r = CLIENT.send(req, HttpResponse.BodyHandlers.ofString());
   Matcher m = Pattern.compile("\"titolo\":\"([^\"]*)\",\"autore\":\"([^\"]*)\",\"anno\":(\\d+)").matcher(r.body()); %>
<!doctype html>
<html lang="it"><head><meta charset="utf-8"><title>Biblioteca</title></head><body>
<h1>Libri della biblioteca</h1>
<table border="1"><tr><th>Titolo</th><th>Autore</th><th>Anno</th></tr>
<% while (m.find()) { %>
  <tr><td><%= html(m.group(1)) %></td><td><%= html(m.group(2)) %></td><td><%= m.group(3) %></td></tr>
<% } %>
</table><p>Risposta dell'API: <%= r.statusCode() %></p></body></html>
```

METTITI ALLA PROVA

Qualcuno aggiunge un libro con titolo `<script>alert(1)</script>`. La JSP senza `html()` …

QUIZ

A Mostra il titolo così com'è, come testo

B Dà errore di compilazione

C Non fa niente: il JSON non può contenere tag

D Manda lo script al browser, che lo esegue

AL COMPUTER · 30 minuti

BiblioAPI in una pagina web, con Tomcat

1. Estrai lo zip di Tomcat 10.1 in **C:\tomcat** (o nella cartella indicata dal docente).
2. Crea la cartella **webapps\biblio** e dentro il file **libri.jsp** della slide.
3. Avvia Tomcat con `bin\startup.bat` e BiblioServer sulla porta 8000.
4. Apri `http://localhost:8080/biblio/libri.jsp`.
5. Aggiungi con curl un libro con titolo `<b>Prova</b>` e ricarica la pagina.

Controllo: una tabella con i libri e in fondo `Risposta dell'API: 200`. Il libro nuovo appare con il testo `<b>Prova</b>` scritto per intero, **non in grassetto**: l'escape funziona. Con "Visualizza sorgente" non c'è traccia del Java.

PUZZLE PER L'ESAME

Il browser chiede libri.jsp: metti in ordine

1. Il browser invia GET /biblio/libri.jsp a Tomcat
2. Tomcat risponde al browser 200 con l'HTML
3. Tomcat traduce e compila la JSP in una servlet, se è la prima volta
4. La servlet scrive le righe della tabella con l'escape
5. BiblioAPI risponde 200 con l'array JSON
6. La servlet invia GET /api/libri a BiblioAPI

MENTI CHE CI HANNO PRECEDUTO

Un livello in più: Tomcat tra il browser e l'API

> **Ogni problema in informatica si può risolvere aggiungendo un livello di indirezione.**
>
> **David Wheeler**, informatico di Cambridge, tra i creatori dell'EDSAC e inventore della subroutine: frase a lui attribuita, resa celebre da Butler Lampson

Il livello in più

La JSP sta in mezzo: il browser non sa che BiblioAPI esiste, e l'API non sa nulla del browser.

Il prezzo

Un salto di rete in più, un server in più da tenere acceso, un punto in più dove qualcosa può guastarsi.

PONTI VERSO OGGI

Un modello linguistico chiamato da Java

```
String chiave = System.getenv("ANTHROPIC_API_KEY");
HttpRequest ai = HttpRequest.newBuilder(
    URI.create("https://api.anthropic.com/v1/messages"))
  .header("x-api-key", chiave)
  .header("anthropic-version", "2023-06-01")
  .header("content-type", "application/json")
  .POST(BodyPublishers.ofString(corpoJson))
  .build();
```





BIGLIETTO D'USCITA

Completa prima di uscire

La classe di Java 11 per fare richieste HTTP si chiama _____.

Se il server è spento, `send` lancia una _____.

Il metodo che non blocca e restituisce un CompletableFuture è _____.

Alla prima richiesta Tomcat traduce la JSP in una _____.

Per evitare il Cross-Site Scripting si fa l'_____ dei caratteri speciali.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- HttpClient, HttpRequest, HttpResponse
- 404 e 500 sono risposte: controlla statusCode()
- Server irraggiungibile o lento: eccezioni e timeout
- sendAsync per chiamate in parallelo
- JSP: Java sul server, HTML al browser
- Ogni dato che finisce nell'HTML va "escapato"

> **All'esame**: un diagramma di sequenza browser → server web → API → database, con i messaggi numerati, spiega un'architettura meglio di una pagina di testo.

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso.